

JavaScript-HTML Interaction Reference Manual

Table of Contents

1. [DOM Fundamentals](#)
2. [Element Selection Methods](#)
3. [Content Manipulation](#)
4. [Attribute Management](#)
5. [Element Creation & Modification](#)
6. [Event Handling](#)
7. [Form Manipulation](#)
8. [Style Manipulation](#)
9. [Navigation & Traversal](#)

DOM Fundamentals

The Document Object Model (DOM) is the bridge between HTML and JavaScript. When a web page loads, the browser creates a DOM tree representing the HTML structure as objects that JavaScript can manipulate.

Key Concepts:

- **Document Object:** The root of the DOM tree (document)
 - **Element Nodes:** HTML tags (<div>, <p>, etc.)
 - **Text Nodes:** Text content within elements
 - **Attribute Nodes:** Element attributes (id, class, etc.)
-

Element Selection Methods

1. getElementById()

Purpose: Select a single element by its unique ID attribute **Syntax:**

`document.getElementById(id)` **HTML Connection:** Targets HTML elements with id attribute

```
// HTML: <div id="myDiv">Content</div>
const element = document.getElementById("myDiv");
```

2. getElementsByClassName()

Purpose: Select multiple elements by class name **Syntax:**

`document.getElementsByClassName(className)` **HTML Connection:** Targets HTML elements with matching class attribute **Returns:** HTMLCollection (live collection)

```
// HTML: <p class="highlight">Text 1</p><p class="highlight">Text 2</p>
const elements = document.getElementsByClassName("highlight");
// elements[0] = first paragraph, elements[1] = second paragraph
```

3. `getElementsByTagName()`

Purpose: Select all elements with specified tag name **Syntax:**

`document.getElementsByTagName(tagName)` **HTML Connection:** Targets HTML elements by tag type **Returns:** HTMLCollection (live collection)

```
// HTML: <p>Para 1</p><p>Para 2</p>
const paragraphs = document.getElementsByTagName("p");
// Selects all <p> elements
```

4. `getElementsByName()`

Purpose: Select elements by name attribute (commonly used with forms)

Syntax: `document.getElementsByName(name)` **HTML Connection:** Targets HTML elements with matching name attribute

```
// HTML: <input type="radio" name="gender" value="male">
const genderInputs = document.getElementsByName("gender");
```

5. `querySelector()`

Purpose: Select first element matching CSS selector **Syntax:**

`document.querySelector(selector)` **HTML Connection:** Uses CSS selector syntax to target HTML elements

```
// HTML: <div class="container"><p id="first">Text</p></div>
const element = document.querySelector("#first"); // By ID
const element2 = document.querySelector(".container p"); // By CSS selector
```

6. `querySelectorAll()`

Purpose: Select all elements matching CSS selector **Syntax:**

`document.querySelectorAll(selector)` **HTML Connection:** Uses CSS selector syntax, returns NodeList

```
// HTML: <div class="item">1</div><div class="item">2</div>
const items = document.querySelectorAll(".item");
// Returns NodeList with all elements having class "item"
```

Content Manipulation

1. innerHTML Property

Purpose: Get or set HTML content inside an element **HTML Connection:**

Directly manipulates the HTML structure within an element

```
// HTML: <div id="container">Original content</div>
const div = document.getElementById("container");

// Get content
console.log(div.innerHTML); // "Original content"

// Set new HTML content
div.innerHTML = "<p>New <strong>HTML</strong> content</p>";
// HTML becomes: <div id="container"><p>New <strong>HTML</strong>
content</p></div>
```

2..textContent Property

Purpose: Get or set text content (strips HTML tags) **HTML Connection:**

Accesses only the text content, ignoring HTML tags

```
// HTML: <p id="text">Hello <strong>World</strong></p>
const paragraph = document.getElementById("text");

// Get text content
console.log(paragraph.textContent); // "Hello World"

// Set text content
paragraph.textContent = "New text content";
// HTML becomes: <p id="text">New text content</p>
```

3. innerText Property

Purpose: Get or set visible text content (respects styling) **HTML**

Connection: Similar to textContent but considers CSS styling

```
// HTML: <div id="hidden" style="display:none">Hidden text</div>
const hiddenDiv = document.getElementById("hidden");

console.log(hiddenDiv.textContent); // "Hidden text"
console.log(hiddenDiv.innerText); // "" (empty because element is
hidden)
```

Attribute Management

1. getAttribute()

Purpose: Get the value of an HTML attribute **HTML Connection:** Reads attribute values from HTML elements

```
// HTML: 
const img = document.getElementById("photo");
```

```
const source = img.getAttribute("src"); // "image.jpg"
const altText = img.getAttribute("alt"); // "Photo"
const category = img.getAttribute("data-category"); // "nature"
```

2. setAttribute()

Purpose: Set or update an HTML attribute **HTML Connection:** Modifies HTML element attributes

```
// HTML: <input id="username" type="text">
const input = document.getElementById("username");

// Set attributes
input.setAttribute("placeholder", "Enter username");
input.setAttribute("maxlength", "20");
input.setAttribute("required", "");

// HTML becomes: <input id="username" type="text" placeholder="Enter
username" maxlength="20" required="">
```

3. removeAttribute()

Purpose: Remove an HTML attribute **HTML Connection:** Completely removes attributes from HTML elements

```
// HTML: <div id="box" class="highlight" style="color:
red;">Content</div>
const box = document.getElementById("box");

box.removeAttribute("style");
// HTML becomes: <div id="box" class="highlight">Content</div>
```

4. hasAttribute()

Purpose: Check if an element has a specific attribute **HTML Connection:** Tests for attribute existence in HTML elements

```
// HTML: <button id="submitBtn" disabled>Submit</button>
const button = document.getElementById("submitBtn");

if (button.hasAttribute("disabled")) {
  console.log("Button is disabled");
}
```

Element Creation & Modification

1. createElement()

Purpose: Create new HTML elements **HTML Connection:** Creates new DOM nodes that can be added to HTML structure

```
// Create new elements
const newDiv = document.createElement("div");
const newParagraph = document.createElement("p");
```

```
const newImg = document.createElement("img");

// Set properties
newDiv.textContent = "New div content";
newParagraph.innerHTML = "This is a <strong>new paragraph</strong>";
newImg.src = "newimage.jpg";
```

2. appendChild()

Purpose: Add a new child element to the end of an element's children

HTML Connection: Modifies HTML structure by adding elements

```
// HTML: <div id="container"></div>
const container = document.getElementById("container");
const newP = document.createElement("p");
newP.textContent = "Added paragraph";

container.appendChild(newP);
// HTML becomes: <div id="container"><p>Added paragraph</p></div>
```

3. insertBefore()

Purpose: Insert a new element before a specified child element **HTML**

Connection: Precisely controls element placement in HTML structure

```
// HTML: <ul id="list"><li>Item 2</li></ul>
const list = document.getElementById("list");
const existingItem = list.querySelector("li");
const newItem = document.createElement("li");
newItem.textContent = "Item 1";

list.insertBefore(newItem, existingItem);
// HTML becomes: <ul id="list"><li>Item 1</li><li>Item 2</li></ul>
```

4. removeChild()

Purpose: Remove a child element **HTML Connection:** Removes elements from HTML structure

```
// HTML: <div id="parent"><p id="child">Remove me</p></div>
const parent = document.getElementById("parent");
const child = document.getElementById("child");

parent.removeChild(child);
// HTML becomes: <div id="parent"></div>
```

5. replaceChild()

Purpose: Replace an existing child element with a new one **HTML**

Connection: Swaps elements in HTML structure

```
// HTML: <div id="container"><p id="old">Old content</p></div>
const container = document.getElementById("container");
const oldElement = document.getElementById("old");
const newElement = document.createElement("h1");
newElement.textContent = "New heading";
```

```
container.replaceChild(newElement, oldElement);  
// HTML becomes: <div id="container"><h1>New heading</h1></div>
```

Event Handling

1. addEventListener()

Purpose: Attach event listeners to HTML elements **HTML Connection:**
Responds to user interactions with HTML elements

```
// HTML: <button id="clickBtn">Click me</button>  
const button = document.getElementById("clickBtn");  
  
button.addEventListener("click", function(event) {  
    console.log("Button was clicked!");  
    event.target.textContent = "Clicked!";  
});
```

2. removeEventListener()

Purpose: Remove previously attached event listeners **HTML Connection:**
Stops responding to HTML element events

```
const button = document.getElementById("clickBtn");  
  
function handleClick() {  
    console.log("Clicked!");  
}  
  
// Add listener  
button.addEventListener("click", handleClick);  
  
// Remove listener  
button.removeEventListener("click", handleClick);
```

3. Event Object Properties

Purpose: Access information about the event and target element **HTML Connection:** Provides data about HTML elements involved in events

```
// HTML: <div id="container"><button>Click me</button></div>  
const container = document.getElementById("container");  
  
container.addEventListener("click", function(event) {  
    console.log("Event type:", event.type);           // "click"  
    console.log("Target element:", event.target);      // The button  
element  
    console.log("Current target:", event.currentTarget); // The  
container element  
    console.log("Target tag name:", event.target.tagName); //  
"BUTTON"  
});
```

4. Common Event Types

- **click**: Mouse click on element
- **submit**: Form submission
- **change**: Input value change
- **keydown/keyup**: Keyboard events
- **mouseover/mouseout**: Mouse hover events
- **load**: Page/element loading complete

```
// HTML: <input id="textInput" type="text">
const input = document.getElementById("textInput");

input.addEventListener("keydown", function(event) {
  console.log("Key pressed:", event.key);
  if (event.key === "Enter") {
    console.log("Enter key was pressed!");
  }
});
```

Form Manipulation

1. Accessing Form Elements

Purpose: Get references to form inputs and controls **HTML Connection:**

Directly interfaces with HTML form elements

```
// HTML: <form id="userForm">
//       <input name="username" type="text">
//       <input name="email" type="email">
//       </form>

const form = document.getElementById("userForm");
const usernameInput = form.elements["username"];
// or
const usernameInput2 = form.querySelector('input[name="username"]');
```

2. Getting/Setting Form Values

Purpose: Read and modify form input values **HTML Connection:** Accesses the value attribute of HTML form elements

```
// HTML: <input id="nameInput" type="text" value="John">
const input = document.getElementById("nameInput");

// Get value
console.log(input.value); // "John"

// Set value
input.value = "Jane";
// HTML attribute becomes: <input id="nameInput" type="text"
value="Jane">
```

3. Form Validation

Purpose: Check form data before submission **HTML Connection:** Uses HTML5 validation attributes and custom validation

```
// HTML: <input id="email" type="email" required>
const emailInput = document.getElementById("email");

function validateEmail() {
  if (emailInput.validity.valid) {
    console.log("Email is valid");
  } else {
    console.log("Email is invalid");
    emailInput.setCustomValidity("Please enter a valid email");
  }
}

emailInput.addEventListener("blur", validateEmail);
```

4. Form Submission Handling

Purpose: Control form submission behavior **HTML Connection:** Intercepts HTML form submission

```
// HTML: <form id="contactForm"><input type="submit" value="Submit">
</form>
const form = document.getElementById("contactForm");

form.addEventListener("submit", function(event) {
  event.preventDefault(); // Prevent default form submission

  // Custom form handling
  const formData = new FormData(form);
  console.log("Form data:", formData);
});
```

Style Manipulation

1. style Property

Purpose: Directly modify CSS styles of HTML elements **HTML Connection:** Changes the style attribute of HTML elements

```
// HTML: <div id="styledDiv">Content</div>
const div = document.getElementById("styledDiv");

// Set individual styles
div.style.backgroundColor = "blue";
div.style.color = "white";
div.style.padding = "10px";
div.style.fontSize = "16px";

// HTML becomes: <div id="styledDiv" style="background-color: blue;
color: white; padding: 10px; font-size: 16px;">Content</div>
```

2. className Property

Purpose: Get or set CSS classes on HTML elements **HTML Connection:**
Modifies the class attribute of HTML elements

```
// HTML: <div id="box" class="container">Content</div>
const box = document.getElementById("box");

// Get classes
console.log(box.className); // "container"

// Set classes
box.className = "container highlight active";
// HTML becomes: <div id="box" class="container highlight
active">Content</div>
```

3. classList Property

Purpose: More convenient way to work with CSS classes **HTML Connection:** Advanced manipulation of the class attribute

```
// HTML: <div id="element" class="original">Content</div>
const element = document.getElementById("element");

// Add class
element.classList.add("new-class");
// HTML: <div id="element" class="original new-class">Content</div>

// Remove class
element.classList.remove("original");
// HTML: <div id="element" class="new-class">Content</div>

// Toggle class
element.classList.toggle("active");
// HTML: <div id="element" class="new-class active">Content</div>

// Check if class exists
if (element.classList.contains("active")) {
  console.log("Element has active class");
}
```

4. getComputedStyle()

Purpose: Get the computed CSS styles of an element **HTML Connection:**
Reads the actual rendered styles of HTML elements

```
// HTML: <div id="styledDiv" style="color: red;">Content</div>
const div = document.getElementById("styledDiv");
const computedStyle = window.getComputedStyle(div);

console.log(computedStyle.color); // "rgb(255, 0, 0)"
console.log(computedStyle.fontSize); // "16px" (default browser
size)
console.log(computedStyle.display); // "block"
```

Navigation & Traversal

1. Parent/Child Relationships

Purpose: Navigate the HTML DOM tree structure **HTML Connection:**

Reflects the hierarchical structure of HTML elements

```
// HTML: <div id="parent">
//      <p id="child1">First child</p>
//      <p id="child2">Second child</p>
//      </div>

const parent = document.getElementById("parent");
const child1 = document.getElementById("child1");

// Access parent
console.log(child1.parentNode); // The div element
console.log(child1.parentElement); // The div element

// Access children
console.log(parent.children); // HTMLCollection of child
elements
console.log(parent.firstElementChild); // First p element
console.log(parent.lastElementChild); // Second p element
```

2. Sibling Relationships

Purpose: Navigate between sibling HTML elements **HTML Connection:**

Moves between elements at the same HTML hierarchy level

```
// HTML: <div>
//      <p id="first">First</p>
//      <p id="middle">Middle</p>
//      <p id="last">Last</p>
//      </div>

const middle = document.getElementById("middle");

console.log(middle.previousElementSibling); // First p element
console.log(middle.nextElementSibling); // Last p element
```

3. Node vs Element Navigation

Purpose: Distinguish between all nodes (including text) and element nodes

only **HTML Connection:** Accounts for whitespace and text nodes in HTML

```
// HTML: <div id="container">
//      <p>Paragraph</p>
//      <!-- Comment -->
//      <span>Span</span>
//      </div>

const container = document.getElementById("container");

// All child nodes (including text nodes and comments)
console.log(container.childNodes); // NodeList with text,
elements, comments

// Only element nodes
console.log(container.children); // HTMLCollection with only
```

Practical Examples

Example 1: Dynamic Content Updates

```
// HTML: <div id="content">Loading...</div>
//       <button id="updateBtn">Update Content</button>

const contentDiv = document.getElementById("content");
const updateBtn = document.getElementById("updateBtn");

updateBtn.addEventListener("click", function() {
  contentDiv.innerHTML = `
    <h2>Updated Content</h2>
    <p>Content updated at: ${new Date().toLocaleTimeString()}
  </p>
  `;
});
```

Example 2: Form Data Collection

```
// HTML: <form id="userData">
//       <input name="firstName" type="text" placeholder="First
Name">
//       <input name="lastName" type="text" placeholder="Last
Name">
//       <button type="submit">Save</button>
//     </form>

const form = document.getElementById("userData");

form.addEventListener("submit", function(event) {
  event.preventDefault();

  const firstName = form.elements["firstName"].value;
  const lastName = form.elements["lastName"].value;

  console.log("User data:", { firstName, lastName });
});
```

Example 3: Interactive List Management

```
// HTML: <ul id="itemList"></ul>
//       <input id="newItem" type="text" placeholder="Add item">
//       <button id="addBtn">Add Item</button>

const itemList = document.getElementById("itemList");
const newItemInput = document.getElementById("newItem");
const addBtn = document.getElementById("addBtn");

function addItem() {
  const itemText = newItemInput.value.trim();
  if (itemText) {
    const li = document.createElement("li");
    li.textContent = itemText;
  }
}
```

```

        // Add delete button
        const deleteBtn = document.createElement("button");
        deleteBtn.textContent = "Delete";
        deleteBtn.addEventListener("click", function() {
            itemList.removeChild(li);
        });

        li.appendChild(deleteBtn);
        itemList.appendChild(li);
        newItemInput.value = "";
    }
}

addBtn.addEventListener("click", addItem);
newItemInput.addEventListener("keydown", function(event) {
    if (event.key === "Enter") {
        addItem();
    }
});

```

Summary

JavaScript provides powerful methods to interact with HTML elements through the DOM. The key concepts are:

1. **Selection:** Use various methods to find HTML elements
2. **Manipulation:** Change content, attributes, and structure
3. **Events:** Respond to user interactions
4. **Traversal:** Navigate the HTML structure
5. **Dynamic Creation:** Build new HTML elements programmatically

Each JavaScript DOM method has a direct relationship with HTML, allowing you to read existing HTML properties and attributes, modify them dynamically, and create entirely new HTML structures that respond to user interactions.